MODULE-1

FUNDAMENTALS JAVA PROGRAMMING

Java is a class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is intended to let application developers Write Once and Run Anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation.

Java is a multi-platform, object-oriented, and network-centric language that can be used as a platform in itself. It is a fast, secure, reliable programming language for coding everything from mobile apps and enterprise software to big data applications and server-side technologies.

The Key Attributes of Object-Oriented Programming

The core idea of OOPs is to bind data and the functions that operate on it, preventing unauthorized access from other parts of the code. Java strictly follows the DRY (Don't Repeat Yourself) Principle, ensuring that common logic is written once (e.g., in parent classes or utility methods) and reused throughout the application. This makes the code:

- **Easier to maintain:** Changes are made in one place.
- **More organized:** Follows a structured approach.
- **Easier to debug and understand:** Reduces redundancy and improves readability.

OOPS stands for **Object-Oriented Programming** System. It is a programming approach that organizes code into objects and classes and makes it more structured and easier to manage. A class is a blueprint that defines properties and behaviors, while an object is an instance of a class representing real-world entities.

Example:

```
// Use of Object and Classes in Java

import java.io.*;

class Numbers {

    // Properties
```

```
    private int a;
    private int b;
    // Setter methods
    public void setA(int a) { this.a = a; }
    public void setB(int b) { this.b = b; }
    // Methods
    public void sum() { System.out.println(a + b); }
    public void sub() { System.out.println(a - b); }
    public static void main(String[] args)
    {
        Numbers obj = new Numbers();
        // Using setters instead of direct access
        obj.setA(1);
        obj.setB(2);
        obj.sum();
        obj.sub();
    }
}
```

It is a simple example showing a class Numbers containing two variables which can be accessed and updated only by instance of the object created.

>>Java Class

A Class is a **user-defined blueprin**t or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times. This includes classes for objects occurring more than once in your code. In general, class declarations can include these components in order:

- **Modifiers**: A class can be public or have default access (Refer to this for details).
- **Class name:** The class name should begin with the initial letter capitalized by convention.
- **Body:** The class body is surrounded by braces, { }.

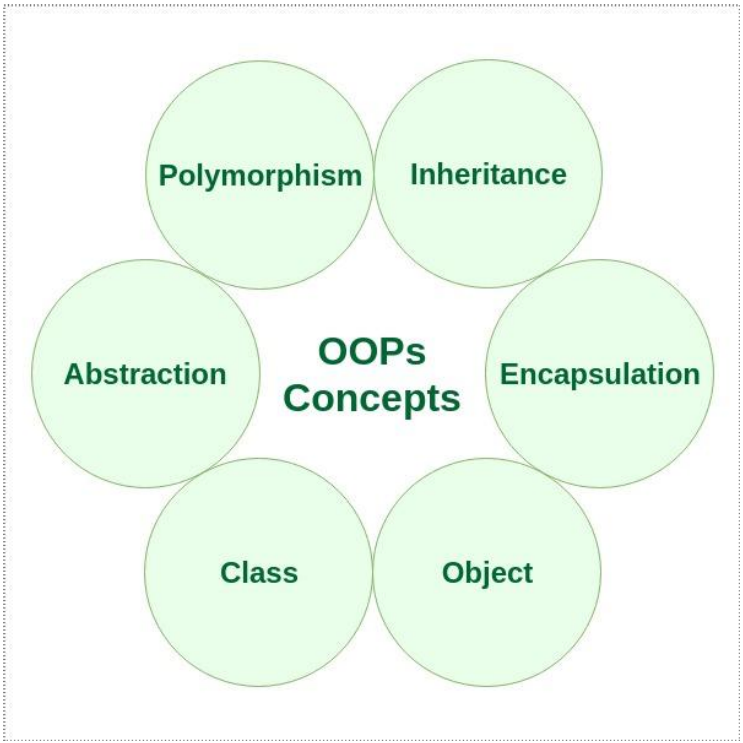
>>Java Object

An Object is a basic unit of Object-Oriented Programming that represents real-life entities. A typical Java program creates many objects, which as you know, interact by invoking methods. The objects are what perform your code, they are the part of your code visible to the viewer/user. An object mainly consists of:

- **State**: It is represented by the attributes of an object. It also reflects the properties of an object.
- **Behaviour**: It is represented by the methods of an object. It also reflects the response of an object to other objects.
- **Identity**: It is a unique name given to an object that enables it to interact with other objects.

>>**Method:** A method is a collection of statements that perform some specific task and return the result to the caller. A method can perform some specific task without returning anything. Methods allow us to **reuse** the code without retyping it, which is why they are considered **time savers**. In Java, every method must be part of some class, which is different from languages like C, C++, and Python.

>>The below diagram demonstrates the Java OOPs Concepts



>>4 Pillars of Java OOPs Concepts

1. Abstraction

Data Abstraction is the property by virtue of which **only the essential details are displayed to the user.** The trivial or non-essential units are not displayed to the user. Data Abstraction may also be defined as the process of identifying only the required characteristics of an object, ignoring the irrelevant details. The properties and behaviors of an object differentiate it from other objects of similar type and also help in classifying/grouping the object.

Real-life Example: Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the car speed or applying brakes will stop the car, but he does not know how on pressing the accelerator, the speed is actually increasing. He does not know about the inner mechanism of the car or the implementation of the accelerators, brakes etc. in the car. This is what abstraction is.

Note: In Java, abstraction is achieved by interfaces and abstract classes. We can achieve 100% abstraction using interfaces.

Example:

```
// Abstract class representing a Vehicle (hiding implementation details)
abstract class Vehicle {
    // Abstract methods (what it can do)
    abstract void accelerate();
    abstract void brake();
        // Concrete method (common to all vehicles)
    void startEngine() {
        System.out.println("Engine started!");
    }
}
// Concrete implementation (hidden details)
class Car extends Vehicle {
    @Override
    void accelerate() {
        System.out.println("Car: Pressing gas pedal...");
        // Hidden complex logic: fuel injection, gear shifting, etc.
    }
        @Override
    void brake() {
```

```
        System.out.println("Car: Applying brakes...");

        // Hidden logic: hydraulic pressure, brake pads, etc.

    }

}

public class Main {

    public static void main(String[] args) {

        Vehicle myCar = new Car();

        myCar.startEngine();

        myCar.accelerate();

        myCar.brake();

    }

}
```

2. Encapsulation

It is defined as the **wrapping up of data under a single unit.** It is the mechanism that binds together the code and the data it manipulates. Another way to think about encapsulation is that it is a protective shield that prevents the data from being accessed by the code outside this shield.

- Technically, in encapsulation, the variables or the data in a class is hidden from any other class and can be accessed only through any member function of the class in which they are declared.
- In encapsulation, the data in a class is hidden from other classes, which is similar to what **data-hiding** does. So, the terms "encapsulation" and "data-hiding" are used interchangeably.

- Encapsulation can be achieved by declaring all the variables in a class as private and writing public methods in the class to set and get the values of the variables.

```
// Encapsulation using private modifier

class Employee {

    // Private fields (encapsulated data)

    private int id;

    private String name;

    // Setter methods

    public void setId(int id) {

        this.id = id;

    }

    public void setName(String name) {

        this.name = name;

    }

    // Getter methods

    public int getId() {

        return id;

    }

    public String getName() {

        return name;

    }
```

```
}

public class Main {

    public static void main(String[] args) {

        Employee emp = new Employee();

            // Using setters

        emp.setId(101);

        emp.setName("Geek");

        // Using getters

        System.out.println("Employee ID: " + emp.getId());

        System.out.println("Employee Name: " + emp.getName());

    }

}
```

3. Inheritance

Inheritance is an important pillar of OOP (Object Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class. We are achieving inheritance by using **extends** keyword. Inheritance is also known as "**is-a**" relationship.

Let us discuss some frequently used important terminologies:

- **Superclass:** The class whose features are inherited is known as superclass (also known as base or parent class).
- **Subclass:** The class that inherits the other class is known as subclass (also known as derived or extended or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.

- **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

```
// Superclass (Parent)
class Animal {
    void eat() {
        System.out.println("Animal is eating...");
    }
    void sleep() {
        System.out.println("Animal is sleeping...");
    }
}
// Subclass (Child) - Inherits from Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking!");
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Dog myDog = new Dog();

        // Inherited methods (from Animal)

        myDog.eat();

        myDog.sleep();

        // Child class method

        myDog.bark();

    }

}
```

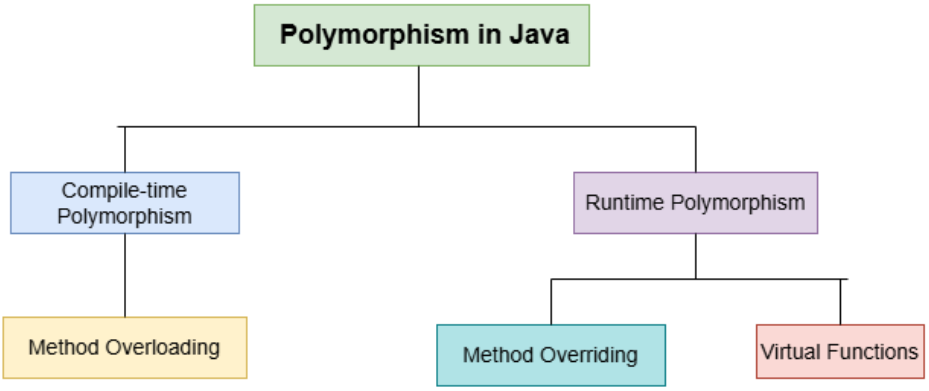
4. Polymorphism

It refers to the ability of object-oriented programming languages t**o differentiate between entities with the same name efficiently**. This is done by Java with the help of the signature and declaration of these entities. The ability to appear in many forms is called polymorphism.

Types of Polymorphism in Java

In Java Polymorphism is mainly divided into two types:

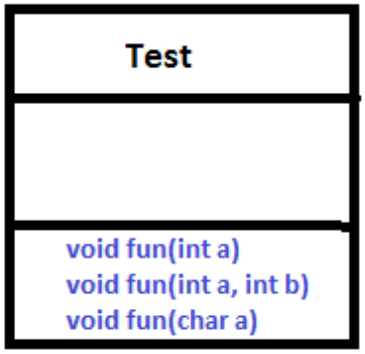
1. **Compile-Time Polymorphism (Static)**
2. **Runtime Polymorphism (Dynamic)**



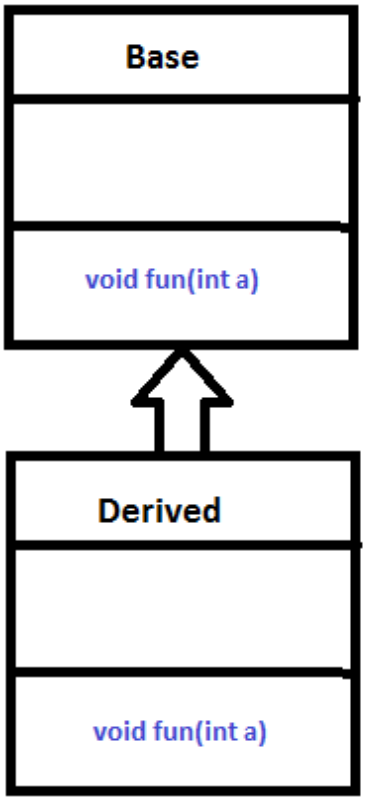
1. Compile-Time Polymorphism

Compile-Time Polymorphism in Java is also known as static polymorphism and also known as method overloading. This happens when multiple methods in the same class have the same name but different parameters.

Note: But Java doesn't support the Operator Overloading.



Overloading



Overriding

Method Overloading

As we discussed above, Method overloading in Java means when there are multiple functions with the same name but different parameters then these functions are said to be overloaded. Functions can be overloaded by changes in the number of arguments or/and a change in the type of arguments.

Example: Method overloading by changing the number of arguments

```
// Java program to demonstrate working of method

// overloading in Java

public class Sum {

      // Overloaded sum()

   // This sum takes two int parameters

   public int sum(int x, int y) { return (x + y); }

   // Overloaded sum()

   // This sum takes three int parameters

   public int sum(int x, int y, int z)

   {

      return (x + y + z);

   }

   // Overloaded sum()

   // This sum takes two double parameters

   public double sum(double x, double y)
```

```
    {
        return (x + y);
    }
        public static void main(String args[])
    {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));
        System.out.println(s.sum(10, 20, 30));
        System.out.println(s.sum(10.5, 20.5));
    }
}
```

Output

30

60

31.0

Different Ways of Method Overloading in Java

The different ways of method overloading in Java are listed below:

- Changing the Number of Parameters.
- Changing Data Types of the Arguments.
- Changing the Order of the Parameters of Methods

1. Changing the Number of Parameters

Method overloading can be achieved by changing the number of parameters while passing to different methods.

Example: This example demonstrates method overloading **by changing the number of parameters in the multiply() method.**

```
// Java Program to Illustrate Method Overloading

// By Changing the Number of Parameters

// Importing required classes

import java.io.*;

// Class 1

// Helper class

class Product {

    // Method 1

   // Multiplying two integer values

   public int multiply(int a, int b)

   {

      int prod = a * b;

      return prod;

   }

   // Method 2

   // Multiplying three integer values
```

```
    public int multiply(int a, int b, int c)
    {
        int prod = a * b * c;
        return prod;
    }
}
// Class 2
// Main class
class Geeks {
        // Main driver method
    public static void main(String[] args)
    {
        // Creating object of above class inside main()
        // method
        Product ob = new Product();
        // Calling method to Multiply 2 numbers
        int prod1 = ob.multiply(1, 2);
        // Printing Product of 2 numbers
        System.out.println(
            "Product of the two integer value: " + prod1);
```

```
        // Calling method to multiply 3 numbers
        int prod2 = ob.multiply(1, 2, 3);
        // Printing product of 3 numbers
        System.out.println(
            "Product of the three integer value: " + prod2);
    }
}
```

Output

Product of the two-integer value: 2

Product of the three-integer value: 6

2. Changing Data Types of the Arguments

In many cases, methods can be considered Overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

Example: This example demonstrates method overloading **by changing the data types of parameters in the Prod() method.**

```
// Java Program to Illustrate Method Overloading
// By Changing Data Types of the Parameters
// Importing required classes
import java.io.*;
// Class 1
// Helper class
```

```
class Product {
        // Multiplying three integer values
    public int Prod(int a, int b, int c)
    {
        int prod1 = a * b * c;
        return prod1;
    }
    // Multiplying three double values.
    public double Prod(double a, double b, double c)
    {
        double prod2 = a * b * c;
        return prod2;
    }
}
class Geeks {
    public static void main(String[] args)
    {
        Product obj = new Product();
        int prod1 = obj.Prod(1, 2, 3);
        System.out.println(
```

```
            "Product of the three integer value: " + prod1);

        double prod2 = obj.Prod(1.0, 2.0, 3.0);

        System.out.println(

            "Product of the three double value: " + prod2);

    }

}
```

Output

Product of the three-integer value: 6

Product of the three double values: 6.0

3. Changing the Order of the Parameters of Methods

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods. For example, if the parameters of method 1 are (String name, int roll_no) and the other method is (int roll_no, String name) but both have the same name, then these 2 methods are considered to be overloaded with different sequences of parameters.

Example: This example demonstrates method overloading **by changing the order of parameters in the StudentId() method.**

```
// Java Program to Illustrate Method Overloading

// By changing the Order of the Parameters

// Importing required classes

import java.io.*;

// Class 1

// Helper class
```

```
class Student {
        // Method 1
    public void StudentId(String name, int roll_no)
    {
        System.out.println("Name: " + name + " "
                           + "Roll-No: " + roll_no);
    }
    // Method 2
    public void StudentId(int roll_no, String name)
    {
        // Again printing name and id of person
        System.out.println("Roll-No: " + roll_no + " "
                           + "Name: " + name);
    }
}
// Class 2
// Main class
class Geeks {
        // Main function
    public static void main(String[] args)
```

```
    {
        // Creating object of above class
        Student obj = new Student();
        // Passing name and id
        // Note: Reversing order
        obj.StudentId("Sweta", 1);
        obj.StudentId(2, "Gudly");
    }
}
```

Output

```
Name: Sweta Roll-No: 1
Roll-No: 2 Name: Gudly
```

Advantages of Method Overloading

The advantages of method overlaoding is listed below:

- Method overloading improves the Readability and reusability of the program.
- Method overloading reduces the complexity of the program.
- Using method overloading, programmers can perform a task efficiently and effectively.
- Using method overloading, it is possible to access methods performing related functions with slightly different arguments and types.
- Objects of a class can also be initialized in different ways using the constructors.

Disadvantages of Method Overloading

- Too many overloaded methods can make the code harder to read and maintain.
- Similar method names with different parameters can confuse developers and lead to incorrect usage.
- Improper use may lead to ambiguity in method selection, especially with type conversions.

2. Runtime Polymorphism

Runtime Polymorphism in Java known as Dynamic Method Dispatch. It is a process in which a function call to the overridden method is resolved at Runtime. This type of polymorphism is achieved by Method Overriding. Method overriding, on the other hand, occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be overridden.

Method Overriding

Method overriding in Java means when a subclass provides a specific implementation of a method that is already defined in its superclass. The method in the subclass must have the same name, return type, and parameters as the method in the superclass. Method overriding allows a subclass to modify or extend the behavior of an existing method in the parent class. This enables dynamic method dispatch, where the method that gets executed is determined at runtime based on the object's actual type.

Example: This program demonstrates method overriding in Java, where the Print() method is redefined in the subclasses (subclass1 and subclass2) to provide specific implementations.

```
// Java Program for Method Overriding

// Class 1

// Helper class

class Parent {
```

```
    // Method of parent class
    void Print() {
        System.out.println("parent class");
    }
}
// Class 2
// Helper class
class subclass1 extends Parent {
    // Method
    void Print() {
      System.out.println("subclass1");
    }
}
// Class 3
// Helper class
class subclass2 extends Parent {
    // Method
    void Print() {
        System.out.println("subclass2");
    }
```

```
}

// Class 4

// Main class

class Geeks {

    // Main driver method

    public static void main(String[] args) {

        // Creating object of class 1

        Parent a;

        // Now we will be calling print methods

        // inside main() method

        a = new subclass1();

        a.Print();

        a = new subclass2();

        a.Print();

    }

}
```

Output

subclass1

subclass2

Explanation: In the above example, when an object of a child class is created, then the method inside the child class is called. This is because the method in the parent class is overridden by

the child class. This method has more priority than the parent method inside the child class. So, the body inside the child class is executed.

Advantages of Polymorphism

- Encourages code reuse.
- Simplifies maintenance.
- Enables dynamic method dispatch.
- Helps in writing generic code that works with many types.

Disadvantages of Polymorphism

- It ca make more difficult to understand the behavior of an object.
- This may cause performance issues, as polymorphic behavior may require additional computations at runtime.

Method Overloading vs Method Overriding

The table below demonstrates the difference between Method Overloading and Method Overriding

Feature	Method Overloading	Method Overriding
Definition	Same method name, different parameters	Same method signature, different class
Polymorphism Type	Compile-time polymorphism (Static)	Runtime polymorphism (Dynamic)
Inheritance	No inheritance involved	Involves inheritance (parent-child)

Advantage of OOPs over Procedure-Oriented Programming Language

Object-oriented programming (OOP) offers several key advantages over procedural programming:

- By using objects and classes, you can create reusable components, leading to less duplication and more efficient development.
- It provides a clear and logical structure, making the code easier to understand, maintain, and debug.
- OOP supports the DRY (Don't Repeat Yourself) principle.This principle encourages minimizing code repetition, leading to cleaner, more maintainable code. Common functionalities are placed in a single location and reused, reducing redundancy.
- By reusing existing code and creating modular components, OOP allows for quicker and more efficient application development

Disadvantages of OOPs

- OOP has concepts like classes, objects, inheritance etc. For beginners, this can be confusing and takes time to learn.
- If we write a small program, using OOP can feel too heavy. We might have to write more code than needed just to follow the OOP structure.
- The code is divided into different classes and layers, so in this, finding and fixing bugs can sometimes take more time.
- OOP creates a lot of objects, so it can use more memory compared to simple programs written in a procedural way.

A First Simple Program

Steps to Implement a Java Program

The implementation of a Java program involves the following steps. They include:

- Creating the program
- Compiling the program
- Running the program

1. Create a Java Program

Java programs can be written in a text editor (Notepad, VS Code) or an IDE (IntelliJ, Eclipse, NetBeans).

```
// Simple Java Hello World Program
public class HelloWorld
{
public static void main(String[] args)
{
System.out.println(“Hello, World”);
}
}
```

Note: Save the file as **HelloWorld.java**

2. Compile the Java Program

To compile the program, we must run the Java compiler (javac), with the name of the source file on the “command prompt” (Windows) or “Terminal” (Mac/Linux) as follows:

javac HelloWorld.java

Note:

- Before running the command, make sure you navigate to the correct directory where your HelloWorld.java file is saved.

- If everything is OK, the compiler creates a file called HelloWorld.class which contains the byte code of the program.

3. Run the Java Program

We need to use the Java Interpreter to run a program. Execute and compile Java program with below command:

java HelloWorld

Output:

Hello, World

Example:

```
// Simple Java program

// FileName: "HelloWorld.java"

public class HelloWorld {

    // Your program begins with a call to main().

  // Prints "Hello, World" to the terminal window.

  public static void main(String[] args)

  {

    System.out.println("Hello, World");

  }

}
```

Output

Hello, World

Understanding the Java Hello World Code

1. Class Definition

Every Java program must have at least one class. This line uses the keyword **class** to declare that a new class is being defined.

class HelloWorld {
//
//Statements
}

Note: If the class is **public**, the **filename must match the class name HelloWorld.java**

2. HelloWorld

It is an identifier that is the name of the class. The entire class definition, including all of its members, will be between the opening curly brace ” { ” and the closing curly brace ” } “.

3. main Method

In the Java programming language, every application must contain a main method. The main function(method) is the entry point of your Java application, and it’s mandatory in a Java program.

Signature of main() Method

public static void main(String[] args)

Explanation of the above syntax:

- **public**: So that JVM can execute the method from anywhere. If we declare something as **public** it simply means it is **accessible from anywhere.**
- **static**: The main method is to be called without creating an object. The modifiers are public and static can be written in either order.
- **void**: The main method doesn’t return any value.

- **main()**: Name configured in the JVM. The main method must be inside the class definition. The compiler executes the codes starting always from the main function.

- **String[]**: The main method accepts a single argument, i.e., an array of elements of type String.

Like in C/C++, the main method is the entry point for your application and will subsequently invoke all the other methods required by your program.

The next line of code is shown here. Notice that it occurs inside the main() method.

System.out.println("Hello, World");

Explanation of the above syntax:

This line is used to print the Hello, World on the console Here is the brief description of the above code:

- **System:** It is the predefined class which is present in the java.lang package which provides the system resources for input and output.

- **out**: It is a static field of type PrintStream in the System class used to represent the standard output stream on the console.

- **. (dot)**: It is the member access operator also known as link operator used to access the members of class or objects i.e fields and methods.

- **println()**: It is the method of PrintStream class which is used to print the message on the new line written inside as string"" (enclosed with double quotes).

Comments

They can either be multiline or single-line comments.

// Simple Java program
// FileName: "HelloWorld.java"

This is a single-line comment. This type of comment must begin with // as in C/C++.

For multiline comments, they must begin from /* and end with */.

/*

This is a

multi-line comment

*/

Important Points:

- The name of the class is HelloWorld, which is same as the name of the file "HelloWorld.java". This is not a coincidence. In Java, all codes must present inside a class, and there is at most one public class which contains the main() method.
- By convention, the name of the main class(a class that contains the main method) should match the name of the file that holds the program.
- Every Java program must have a class definition that matches the filename (class name and file name should be same).

Compiling the Program

After successfully setting up the environment, we can open a terminal in both Windows/Unix and go to the directory where the file – HelloWorld.java is present. Now, to compile the HelloWorld program, execute the compiler – javac, to specify the name of the source file on the command line, as shown:

javac HelloWorld.java

The compiler creates a HelloWorld.class (in the current working directory) that contains the bytecode version of the program. Now, to execute our program, JVM (Java Virtual Machine) needs to be called using Java, specifying the name of the class file on the command line, as shown:

java HelloWorld

Handling Syntax Errors

Error is an illegal operation performed by the user which results in the abnormal working of the program. Programming errors often remain undetected until the program is compiled or executed. Some of the errors inhibit the program from getting compiled or executed. Thus errors should be removed before compiling and executing.

Types of Errors

The errors can be broadly classified into three categories: Runtime Errors, Compile-Time Errors, and Logical Errors. Each type of error has distinct characteristics and requires specific methods for detection and resolution.

1. Run Time Error

Run Time errors occur or we can say, are detected during the execution of the program. Sometimes these are discovered when the user enters an invalid data or data which is not relevant. Runtime errors occur when a program does not contain any syntax errors but asks the computer to do something that the computer is unable to reliably do. During compilation, the compiler has no technique to detect these kinds of errors. It is the JVM (Java Virtual Machine) that detects it while the program is running. To handle the error during the run time we can put our error code inside the try block and catch the error inside the catch block.

For example: if the user inputs a data of string format when the computer is expecting an integer, there will be a runtime error.

Example 1: Runtime Error caused by dividing by zero

```
// Java program to demonstrate Runtime Error

class DivByZero {

   public static void main(String args[])

   {

      int var1 = 15;
```

```
        int var2 = 5;

        int var3 = 0;

        int ans1 = var1 / var2;

        // This statement causes a runtime error,

        // as 15 is getting divided by 0 here

        int ans2 = var1 / var3;

        System.out.println(  "Division of va1"   + " by var2 is: "    + ans1);

        System.out.println(  "Division of va1" + " by var3 is: " + ans2);

    }

}
```

Runtime Error in java code:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at DivByZero.main(File.java:14)
```

2. Compile Time Error

Compile Time Errors are those errors which prevent the code from running because of an incorrect syntax such as a missing semicolon at the end of a statement or a missing bracket, class not found, etc. These errors are detected by the java compiler and an error message is displayed on the screen while compiling. Compile Time Errors are sometimes also referred to as Syntax errors. These kind of errors are easy to spot and rectify because the java compiler finds them for you. The compiler will tell you which piece of code in the program got in trouble and its best guess as to what you did wrong. Usually, the compiler indicates the exact line where the error is, or sometimes the line just before it, however, if the problem is with incorrectly nested braces, the actual error may be at the beginning of the block. In effect, syntax errors represent grammatical errors in the use of the programming language.

Example 1: Misspelled variable name or method names

```
class MisspelledVar {
    public static void main(String args[])
    {
        int a = 40, b = 60;
        // Declared variable Sum with Capital S
        int Sum = a + b;
        // Trying to call variable Sum
        // with a small s ie. sum
        System.out.println(
            "Sum of variables is "
            + sum);
    }
}
```

Compilation Error in java code:

```
prog.java:14: error: cannot find symbol
            + sum);
              ^
  symbol:   variable sum
  location: class MisspelledVar
1 error
```

3. Logical Error

A logic error is when your program compiles and executes, but does the wrong thing or returns an incorrect result or no output when it should be returning an output. These errors are detected neither by the compiler nor by JVM. The Java system has no idea what your program is supposed to do, so it provides no additional information to help you find the error. Logical errors are also called Semantic Errors. These errors are caused due to an incorrect idea or concept used by a programmer while coding. Syntax errors are grammatical errors whereas, logical errors are errors arising out of an incorrect meaning. For example, if a programmer accidentally adds two variables when he or she meant to divide them, the program will give no error and will execute successfully but with an incorrect result.

Example: Displaying the wrong message

```
class IncorrectMessage {

   public static void main(String args[])

   {

      int a = 2, b = 8, c = 6;

      System.out.println("Finding the largest number \n");

      if (a > b && a > c)

         System.out.println(  a + " is the largest Number");

      else if (b > a && b > c)

         System.out.println(b + " is the smallest Number");

      // The correct message should have

      // been System.out.println

      //(b+" is the largest Number");
```

```
        // to make logic
        else
            System.out.println(c + " is the largest Number");
    }
}
```

Output

Finding the largest number

8 is the smallest Number

4. Syntax Error

Syntax and Logical errors are faced by Programmers.

Spelling or grammatical mistakes are syntax errors, for example, using an uninitialized variable, using an undefined variable, etc., missing a semicolon, etc.

```
int x, y;
x = 10 // missing semicolon (;)
z = x + y; // z is undefined, y in uninitialized.
```

Syntax errors can be removed with the help of the compiler.

>>Introducing Data Types and Operators

>**Data Types**

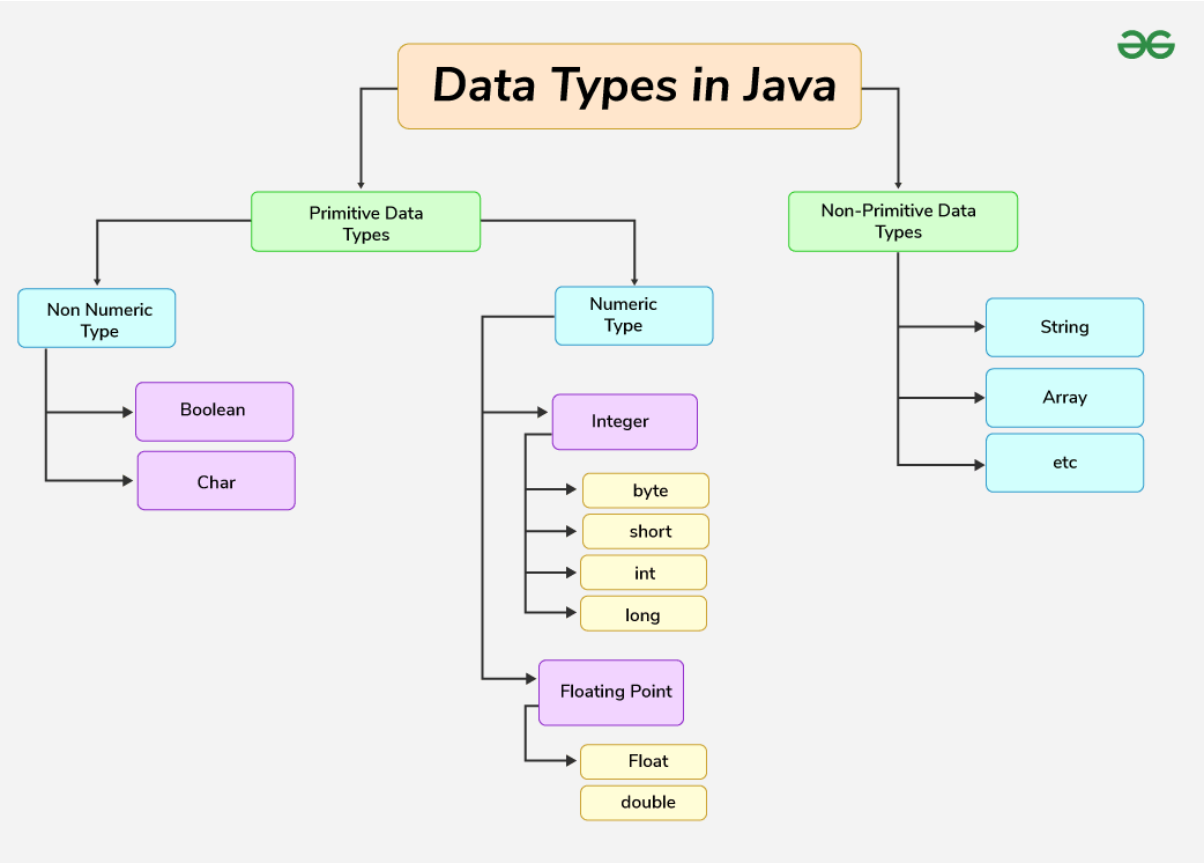
A **data type** is a classification that specifies which type of value a variable can hold. It determines the kind of data (such as integers, floating-point numbers, characters, or objects) and the operations that can be performed on it.

A data type in Java defines the size and type of variable values and determines the kind of operations that can be performed on those values.

Why Data Types Matter in Java?

Data types matter in Java because of the following reasons, which are listed below:

- **Memory Efficiency:** Choosing the right type (byte vs int) saves memory.
- **Performance:** Proper types reduce runtime errors.
- **Code Clarity:** Explicit typing makes code more readable.



Java Data Type Categories

Java has two categories in which data types are segregated

- **Primitive Data Type:** These are the basic building blocks that store simple values directly in memory. Examples of primitive data types are **boolean, char, byte, short, int, long, float, and double.**

- **Non-Primitive Data Types (Object Types):** These are reference types that store memory addresses of objects. Examples of non-primitive data types are Array, Class, Interface, and Object.

Primitive Data Types

1. boolean Data Type

The boolean data type represents a logical value that can be either true or false. Conceptually, it represents a single bit of information, but the actual size used by the virtual machine is implementation-dependent and typically at least one byte (eight bits) in practice. Values of the boolean type are not implicitly or explicitly converted to any other type using casts. However, programmers can write conversion code if needed.

Syntax:

boolean booleanVar;

Example:

```
// Demonstrating boolean data type

public class Geeks {

    public static void main(String[] args) {

        boolean b1 = true; boolean b2 = false;

        System.out.println("Is Java fun? " + b1);

        System.out.println("Is fish tasty? " + b2);

    }

}
```

Output:

```
Is Java fun? true

Is fish tasty? false
```

2. byte Data Type

The byte data type is an 8-bit signed two’s complement integer. The byte data type is useful for saving memory in large arrays.

Syntax:

byte byteVar;

Example:

```
// Demonstrating byte data type

public class Geeks {

    public static void main(String[] args) {

        byte a = 25;

        byte t = -10;

        System.out.println("Age: " + a);

        System.out.println("Temperature: " + t);

    }

}
```

Output:

Age: 25

Temperature: -10

3. short Data Type

The short data type is a 16-bit signed two’s complement integer. Similar to byte, a short is used when memory savings matter, especially in large arrays where space is constrained.

Syntax:

short shortVar;

Example:

```
// Demonstrating short data types

public class Geeks {

    public static void main(String[] args) {

        short num = 1000;

        short t = -200;

        System.out.println("Number of Students: " + num);

        System.out.println("Temperature: " + t);

    }

}
```

Output:

Number of Students: 1000

Temperature: -200

4. int Data Type

It is a 32-bit signed two’s complement integer.

Syntax:

int intVar;

Example:

```
// Demonstrating int data types

public class Geeks {

    public static void main(String[] args) {

        int p = 2000000;

        int d = 150000000;

        System.out.println("Population: " + p);

        System.out.println("Distance: " + d);

    }

}
```

Output:

Population: 2000000

Distance: 150000000

5. long Data Type

The long data type is a 64-bit signed two’s complement integer. It is used when an int is not large enough to hold a value, offering a much broader range.

Syntax:

long longVar;

Example:

```
// Demonstrating long data type

public class Geeks {

    public static void main(String[] args) {

        long w = 7800000000L;

        long l = 9460730472580800L;

        System.out.println("World Population: " + w);

        System.out.println("Light Year Distance: " + l);

    }

}
```

Output:

World Population: 7800000000

Light Year Distance: 9460730472580800

6. float Data Type

The float data type is a single-precision 32-bit IEEE 754 floating-point. Use a float (instead of double) if you need to save memory in large arrays of floating-point numbers. The size of the float data type is 4 bytes (32 bits).

Syntax:

float floatVar;

Example:

```
// Demonstrating float data type

public class Geeks {

    public static void main(String[] args) {

        float pi = 3.14;

        float gravity = 9.81;

        System.out.println("Value of Pi: " + pi);

        System.out.println("Gravity: " + gravity);

    }

}
```

Output:

Value of Pi: 3.14

Gravity: 9.81

7. double Data Type

The double data type is a double-precision 64-bit IEEE 754 floating-point. For decimal values, this data type is generally the default choice. The size of the double data type is 8 bytes or 64 bits.

Syntax:

double doubleVar;

Example:

```
// Demonstrating double data type

public class Geeks {

    public static void main(String[] args) {

        double pi = 3.141592653589793;

        double an = 6.02214076e23;

        System.out.println("Value of Pi: " + pi);

        System.out.println("Avogadro's Number: " + an);

    }

}
```

Output:

Value of Pi: 3.141592653589793

Avogadro's Number: 6.02214076E23

8. char Data Type

The char data type is a single 16-bit Unicode character with the size of 2 bytes (16 bits).

Syntax:

char charVar;

Example:

```
// Demonstrating char data type

public class Geeks{

    public static void main(String[] args) {

        char g = 'A';

        char s = '$';

        System.out.println("Grade: " + g);

        System.out.println("Symbol: " + s);

    }

}
```

Output:

Grade: A

Symbol: $

>> Write a Java Program to Demonstrate Primitive Data Types in Java

```
// Java Program to Demonstrate Char Primitive Data Type

class Primitive

{

    public static void main(String args[])

    {



        char a = 'G';

        int i = 89;

        byte b = 4;

        short s = 56;

        double d = 4.355453532;

        float f = 4.7333434f;

        long l = 12121;

        System.out.println("char: " + a);

        System.out.println("integer: " + i);

        System.out.println("byte: " + b);

        System.out.println("short: " + s);

        System.out.println("float: " + f);

        System.out.println("double: " + d);
```

```
        System.out.println("long: " + l);
    }
}
```

Output:

char: G

integer: 89

byte: 4

short: 56

float: 4.7333436

double: 4.355453532

long: 12121

Non-Primitive (Reference) Data Types

The **Non-Primitive (Reference) Data Types** will contain a memory address of variable values because the reference types won't store the variable value directly in memory. They are strings, objects, arrays, etc.

1. Strings

Strings are defined as an array of characters. The difference between a character array and a string in Java is, that the string is designed to hold a sequence of characters in a single variable whereas, a character array is a collection of separate char-type entities. Unlike C/C++, Java strings are not terminated with a null character.

Syntax: Declaring a string

<String_Type> <string_variable> = "<sequence_of_string>";

Example:

```
// Demonstrating String data type

public class Geeks {

    public static void main(String[] args) {

        String n = "Geek1";

        String m = "Hello, World!";

        System.out.println("Name: " + n);

        System.out.println("Message: " + m);

    }

}
```

Output:

```
Name: Geek1

Message: Hello, World!
```

2. Class

A Class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. In general, class declarations can include these components, in order:

- **Modifiers** : A class can be public or has default access. Refer to access specifiers for classes or interfaces in Java
- **Class name:** The name should begin with an initial letter (capitalized by convention).

- **Superclass(if any):** The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.
- **Interfaces(if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.
- **Body:** The class body is surrounded by braces, { }.

Example:

```
// Demonstrating how to create a class

class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
    void display() {
        System.out.println(model + " " + year);
    }
}



public class Geeks {
```

```
    public static void main(String[] args) {
        Car myCar = new Car("Toyota", 2020);
        myCar.display();
    }
}
```

Output:

Toyota 2020

3. Object

An Object is a basic unit of Object-Oriented Programming and represents real-life entities. A typical Java program creates many objects, which as you know, interact by invoking methods. An object consists of :

- **State** : It is represented by the attributes of an object. It also reflects the properties of an object.
- **Behavior** : It is represented by the methods of an object. It also reflects the response of an object to other objects.
- **Identity** : It gives a unique name to an object and enables one object to interact with other objects.

Example:

```
// Define the Car class
class Car {
    String model;
```

```
    int year;


    // Constructor to initialize the Car object
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
}
// Main class to demonstrate object creation
public class Geeks {
    public static void main(String[] args) {
        // Create an object of the Car class
        Car myCar = new Car("Honda", 2021);
        System.out.println("Car Model: " + myCar.model);
        System.out.println("Car Year: " + myCar.year);
    }
}
```

Output:

```
Car Model: Honda
Car Year: 2021
```

4. Interface

Like a class, an interface can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, no body).

- Interfaces specify what a class must do and not how. It is the blueprint of the class.
- An Interface is about capabilities like a Player may be an interface and any class implementing Player must be able to (or must implement) move(). So it specifies a set of methods that the class has to implement.
- If a class implements an interface and does not provide method bodies for all functions specified in the interface, then the class must be declared abstract.
- A Java library example is Comparator Interface. If a class implements this interface, then it can be used to sort a collection.

Example:

```
// Demonstrating the working of interface

interface Animal {

   void sound();

}

class Dog implements Animal {

   public void sound() {

      System.out.println("Woof");

   }

}

public class InterfaceExample {
```

```
    public static void main(String[] args) {

        Dog myDog = new Dog();

        myDog.sound();

    }

}
```

Output:

Woof

5. Array

An Array is a group of like-typed variables that are referred to by a common name. Arrays in Java work differently than they do in C/C++. The following are some important points about Java arrays.

- In Java, all arrays are dynamically allocated. (discussed below)
- Since arrays are objects in Java, we can find their length using member length. This is different from C/C++ where we find length using size.
- A Java array variable can also be declared like other variables with [] after the data type.
- The variables in the array are ordered and each has an index beginning with 0.
- Java array can also be used as a static field, a local variable, or a method parameter.
- The **size** of an array must be specified by an int value and not long or short.
- The direct superclass of an array type is Object.
- The length of arrays in Java is fixed once declared, and it can be accessed using the length attribute

Example:

```
// Demonstrating how to create an array

public class Geeks {

    public static void main(String[] args) {

        int[] num = {1, 2, 3, 4, 5};

        String[] arr = {"Geek1", "Geek2", "Geek3"};

        System.out.println("First Number: " + num[0]);

        System.out.println("Second Fruit: " + arr[1]);

    }

}
```

Output:

First Number: 1

Second Fruit: Geek2

> Operators in Java:

Operators are special symbols that perform operations on variables or values. These operators are essential in programming as they allow you to manipulate data efficiently. They can be classified into different categories based on their functionality.

Types of Operators in Java

1. Arithmetic Operators
2. Unary Operators

3. Assignment Operator
4. Relational Operators
5. Logical Operators
6. Ternary Operator
7. Bitwise Operators
8. Shift Operators
9. instance of operator

1. Arithmetic Operators

Arithmetic Operators are used to perform simple arithmetic operations on primitive and non-primitive data types.

- *** :** Multiplication
- **/ :** Division
- **% :** Modulo
- **+ :** Addition
- **– :** Subtraction

Example: This example demonstrates the use of **arithmetic operators on integers and string-to-integer conversion for performing mathematical operations.**

```
import java.io.*;

class Geeks

{

    public static void main (String[] args)
```

```
    {
    // Arithmetic operators on integers
        int a = 10;
        int b = 3;
      // Arithmetic operators on Strings
        String n1 = "15";
        String n2 = "25";
        // Convert Strings to integers
        int a1 = Integer.parseInt(n1);
        int b1 = Integer.parseInt(n2);
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b));
        System.out.println("a % b = " + (a % b));
        System.out.println("a1 + b1 = " + (a1 + b1));
    }
}
```

Output:

a + b = 13

a - b = 7

a * b = 30

a / b = 3

a % b = 1

a1 + b1 = 40

2. Unary Operators

Unary Operators need only one operand. They are used to increment, decrement, or negate a value.

- - , Negates the value.
- + , Indicates a positive value (automatically converts byte, char, or short to int).
- ++ , Increments by 1.
    - Post-Increment: Uses value first, then increments.
    - Pre-Increment: Increments first, then uses value.
- -- , Decrements by 1.
    - Post-Decrement: Uses value first, then decrements.
    - Pre-Decrement: Decrements first, then uses value.

Example: This example demonstrates the use of unary operators for post-increment, pre-increment, post-decrement, and pre-decrement operations.

import java.io.*;

```
// Driver Class
class Geeks {
    // main function
   public static void main(String[] args)
   {
      // Interger declared
      int a = 10;
      int b = 10;
      // Using unary operators
      System.out.println("Postincrement : " + (a++));
      System.out.println("Preincrement : " + (++a));
      System.out.println("Postdecrement : " + (b--));
      System.out.println("Predecrement : " + (--b));
   }
}
```

Output:

```
Postincrement : 10
Preincrement : 12
Postdecrement : 10
Predecrement : 8
```

3. Assignment Operator

'=' Assignment operator is used to assign a value to any variable. It has right-to-left associativity, i.e. value given on the right-hand side of the operator is assigned to the variable on the left, and therefore right-hand side value must be declared before using it or should be a constant.

The general format of the assignment operator is:

variable **=** *value;*

The Assignment Operator is generally of two types. They are:

1. **Simple Assignment Operator:** The Simple Assignment Operator is used with the "=" sign where the left side consists of the operand and the right side consists of a value. The value of the right side must be of the same data type that has been defined on the left side.

   This is the most straightforward assignment operator, which is used to assign the value on the right to the variable on the left. This is the basic definition of an assignment operator and how it functions.

   Syntax:

   num1 = num2;

   Example:

   ```
   import java.io.*;

    class Assignment {

       public static void main(String[] args)
   ```

```
    {

        // Declaring variables

        int num;

        String name;

        num = 10;

        name = "GeeksforGeeks";

        System.out.println("num is assigned: " + num);

        System.out.println("name is assigned: " + name);

    }

}
```

Output:

num is assigned: 10

name is assigned: GeeksforGeeks

2. **Compound Assignment Operator:** The Compound Operator is used where +,-,*, and / is used along with the = operator.

   In many cases, the assignment operator can be combined with others to create shorthand compound statements. For example, a += 5 replaces a = a + 5. Common compound operators include:

- += , Add and assign.
- -= , Subtract and assign.
- *= , Multiply and assign.
- /= , Divide and assign.
- %= , Modulo and assign.

Example: This example demonstrates the use of **various assignment operators, including compound.**

```
import java.io.*;
class Geeks {
    // Main Function
    public static void main(String[] args)
    {
        int f = 7;
        System.out.println("f += 3: " + (f += 3));
        System.out.println("f -= 2: " + (f -= 2));
        System.out.println("f *= 4: " + (f *= 4));
        System.out.println("f /= 3: " + (f /= 3));
        System.out.println("f %= 2: " + (f %= 2));
    }
}
```

Output:

```
f += 3: 10
f -= 2: 8
f *= 4: 32
```

f /= 3: 10

f %= 2: 0

4. Relational Operators

Relational Operators are used to check for relations like equality, greater than, and less than. They return boolean results after the comparison and are extensively used in looping statements as well as conditional if-else statements. The general format is ,

variable ***relation_operator*** *value*

Relational operators compare values and return Boolean results:

- == , Equal to.
- != , Not equal to.
- < , Less than.
- <= , Less than or equal to.
- > , Greater than.
- >= , Greater than or equal to.

Example: This example demonstrates the **use of relational operators to compare values and return boolean results.**

```
// Java Program to show the use of

// Relational Operators

import java.io.*;
```

```
// Driver Class

class Geeks {

        public static void main(String[] args)

  {

        int a = 10;

        int b = 3;

        int c = 5;

        System.out.println("a > b: " + (a > b));

        System.out.println("a < b: " + (a < b));

        System.out.println("a >= b: " + (a >= b));

        System.out.println("a <= b: " + (a <= b));

        System.out.println("a == c: " + (a == c));

        System.out.println("a != c: " + (a != c));

    }

}
```

Output

```
a > b: true

a < b: false

a >= b: true

a <= b: false
```

a == c: false

a != c: true

5. Logical Operators

Logical Operators are used to perform “logical AND” and “logical OR” operations, similar to AND gate and OR gate in digital electronics. They have a short-circuiting effect, meaning the second condition is not evaluated if the first is false.

Conditional operators are:

- **&&, Logical AND:** returns true when both conditions are true.
- **||, Logical OR:** returns true if at least one condition is true.
- **!, Logical NOT:** returns true when a condition is false and vice-versa

Example: This example demonstrates the **use of logical operators (&&, ||, !) to perform boolean operations.**

```
import java.io.*;

class Geeks {

    public static void main (String[] args) {

        // Logical operators

     boolean x = true;

     boolean y = false;

     System.out.println("x && y: " + (x && y));
```

```
        System.out.println("x || y: " + (x || y));

        System.out.println("!x: " + (!x));

    }

}
```

Output

```
x && y: false

x || y: true

!x: false
```

6. Ternary Operator/ Conditional Operator

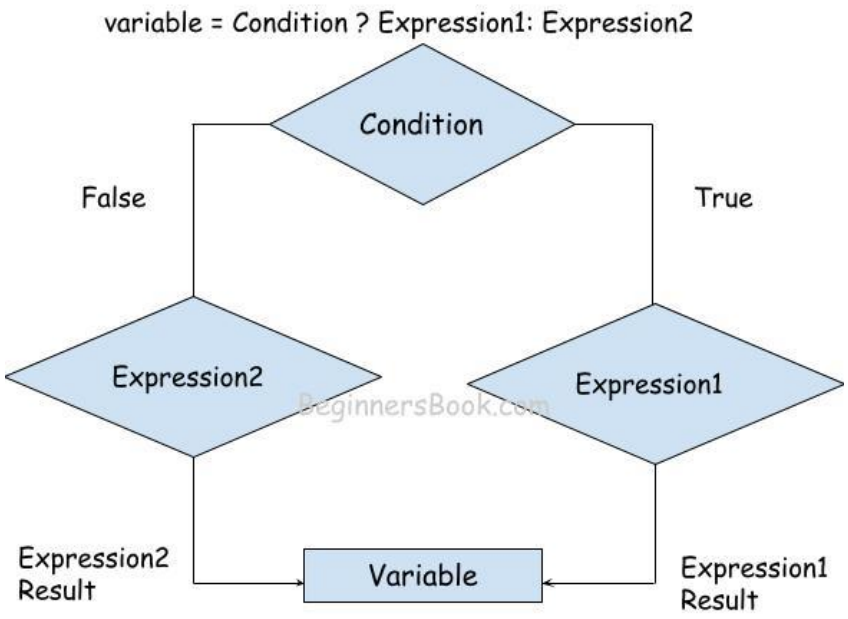
The Ternary Operator is a shorthand version of the if-else statement. It has three operands and hence the name Ternary. The general format is,

variable = Expression1 ? Expression2: Expression3

Note:

- *If Expression1 is **true,** Expression2 is executed*
- *If Expression1 is **false**, Expression3 is executed.*

The above statement means that if the condition evaluates to true, then execute the statements after the '?' else execute the statements after the ':'.



Explanation:

- **Expression1:** The condition is evaluated first (for example, x > 5).
- **Checks True/False:** If the condition is True, it executes Expression2, if False, it runs Expression3.
- **Returns a Result:** The output of either Expression2 or Expression3 becomes the resultant value.
- **Stores the Value:** The final result is assigned to a variable.

Example: This example demonstrates the **use of the ternary operator**

```
import java.io.*;

class Geeks {

    public static void main(String[] args)

    {
```

```
        // variable declaration
        int n1 = 5, n2 = 10, max;
        System.out.println("First num: " + n1);
        System.out.println("Second num: " + n2);
        max = (n1 > n2) ? n1 : n2;


        // Print the largest number
        System.out.println("Maximum is = " + max);
    }
}
```

Output

First num: 5

Second num: 10

Maximum is = 10

7. Bitwise Operators

Bitwise Operators are used to perform the manipulation of individual bits of a number and with any of the integer types. They are used when performing update and query operations of the Binary indexed trees.

- **& (Bitwise AND):** returns bit-by-bit AND of input values.
- **| (Bitwise OR):** returns bit-by-bit OR of input values.
- **^ (Bitwise XOR):** returns bit-by-bit XOR of input values.

- **~ (Bitwise Complement):** inverts all bits (one's complement).

Example: This example demonstrates the **use of bitwise operators (&, |, ^, ~, <<, >>, >>>) to perform bit-level operations.**

1. Bitwise AND (&)

This operator is a binary operator, denoted by '&.' It returns bit by bit AND of input values, i.e., if both bits are 1, it gives 1, else it shows 0.

Example:

a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise AND Operation of 5 and 7
0101
& 0111

_____
0101 = 5 (In decimal)

2. Bitwise OR (|)

This operator is a binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e., if either of the bits is 1, it gives 1, else it shows 0.

Example:

a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise OR Operation of 5 and 7
0101
| 0111

_____
0111 = 7 (In decimal)

3. Bitwise XOR (^)

This operator is a binary operator, denoted by '^.' It returns bit by bit XOR of input values, i.e., if corresponding bits are different, it gives 1, else it shows 0.

Example:

a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise XOR Operation of 5 and 7
0101
^ 0111

_____
0010 = 2 (In decimal)

4. Bitwise Complement (~)

This operator is a unary operator, denoted by '~.' It returns the one's complement representation of the input value, i.e., with all bits inverted, which means it makes every 0 to 1, and every 1 to 0.

Example:

a = 5 = 0101 (In Binary)

Bitwise Complement Operation of 5 in java (8 bits)

~ 00000101

_____
11111010 = -6 (In decimal)

>> Write a Java program demonstrating all bitwise operators.

```
   public class Geeks {

   public static void main(String[] args)

   {

      int a = 5;

      int b = 7;

      System.out.println("a&b = " + (a & b));



      System.out.println("a^b = " + (a ^ b));

      System.out.println("~a = " + ~a);

      a &= b;

      System.out.println("a= " + a);

   }

}
```

Output

```
a&b = 5

a|b = 7

a^b = 2

~a = -6

a= 5
```

8. Shift Operators

Shift Operators are used to shift the bits of a number left or right, thereby multiplying or dividing the number by two, respectively. They can be used when we have to multiply or divide a number by two. The general format ,

*number **shift_op** number_of_places_to_shift;*

- **<< (Left shift):** Shifts bits left, filling 0s (multiplies by a power of two).
- **>> (Signed right shift):** Shifts bits right, filling 0s (divides by a power of two), with the leftmost bit depending on the sign.

Example: This example demonstrates the **use of shift operators (<<, >>) to shift the bits of a number left and right.**

```
import java.io.*;

class Geeks

{

   public static void main(String[] args)

   {

      int a = 10;

         // Using left shift

      System.out.println("a<<1 : " + (a << 1));

          // Using right shift

      System.out.println("a>>1 : " + (a >> 1));

   }
```

}

Output:

a<<1 : 20

a>>1 : 5

9. instanceof Operator

The instanceof operator is used for type checking. It can be used to test if an object is an instance of a class, a subclass, or an interface. The general format,

object ***instance of*** *class/subclass/interface*

Example: This example demonstrates the **use of the instanceof operator to check if an object is an instance of a specific class or interface.**

```
public class Geeks
{
    public static void main(String[] args)
    {

        Person obj1 = new Person();
        Person obj2 = new Boy();

        // As obj is of type person, it is not an
        // instance of Boy or interface
        System.out.println("obj1 instanceof Person: "
                           + (obj1 instanceof Person));
        System.out.println("obj1 instanceof Boy: "
                           + (obj1 instanceof Boy));
        System.out.println("obj1 instanceof MyInterface: "
                           + (obj1 instanceof MyInterface));

        // Since obj2 is of type boy,
        // whose parent class is person
        // and it implements the interface Myinterface
        // it is instance of all of these classes
        System.out.println("obj2 instanceof Person: "
                           + (obj2 instanceof Person));
        System.out.println("obj2 instanceof Boy: "
                           + (obj2 instanceof Boy));
        System.out.println("obj2 instanceof MyInterface: "
                           + (obj2 instanceof MyInterface));
    }
}

// Classes and Interfaces used
// are declared here
class Person {
}

class Boy extends Person implements MyInterface {
}

interface MyInterface {
}
```

Output:

obj1 instanceof Person: true

obj1 instanceof Boy: false

obj1 instanceof MyInterface: false

obj2 instanceof Person: true

obj2 instanceof Boy: true

obj2 instanceof MyInterface: true

>> **Literals in Java**

Any constant value which can be assigned to the variable is called literal/constant.

In simple words, Literals in Java is a synthetic representation of boolean, numeric, character, or string data. It is a medium of expressing particular values in the program, such as an integer variable named ‘’/count is assigned an integer value in the following statement.

Integral literals

For Integral data types (byte, short, int, long), we can specify literals in 4 ways:-

1. Decimal literals (Base 10): In this form, the allowed digits are 0-9.

int x = 101;

2. Octal literals (Base 8): In this form, the allowed digits are 0-7.

// The octal number should be prefix with 0.

int x = 0146;

3. Hexa-decimal literals (Base 16): In this form, the allowed digits are 0-9, and characters are a-f. We can use both uppercase and lowercase characters as we know that java is a case-sensitive programming language, but here java is not case-sensitive.

// The hexa-decimal number should be prefix

// with 0X or 0x.

int x = 0X123Face;

4. Binary literals: From 1.7 onward, we can specify literal value even in binary form also, allowed digits are 0 and 1. Literals value should be prefixed with 0b or 0B.

int x = 0b1111;

Floating-Point literal

For Floating-point data types, we can specify literals in only decimal form, and we cant specify in octal and Hexadecimal forms.

1. Decimal literals(Base 10): In this form, the allowed digits are 0-9.

double d = 123.456;

Char literals

For char data types, we can specify literals in 4 ways:

1. Single quote: We can specify literal to a char data type as a single character within the single quote.

char ch = 'a';

2. Char literal as Integral literal: we can specify char literal as integral literal, which represents the Unicode value of the character, and that integral literal can be specified either in Decimal, Octal, and Hexadecimal forms. But the allowed range is 0 to 65535.

char ch = 062;

3. Unicode Representation: We can specify char literals in Unicode representation '\uxxxx'. Here xxxx represents 4 hexadecimal numbers.

char ch = '\u0061';// Here /u0061 represent a.

4. Escape Sequence: Every escape character can be specified as char literals.

char ch = '\n';

String literals

Any sequence of characters within double quotes is treated as String literals.

String s = "Hello";

String literals may not contain unescaped newline or linefeed characters. However, the Java compiler will evaluate compile-time expressions, so the following String expression results in a string with three lines of text:

Example:

String text = "This is a String literal\n"

+ "which spans not one and not two\n"

+ "but three lines of text.\n";

Boolean literals

Only two values are allowed for Boolean literals, i.e., true and false.

boolean b = true;

>> **Java Variables**

In Java, variables are containers that store data in memory. Understanding variables plays a very important role as it defines how data is stored, accessed, and manipulated.

Key Components of Variables in Java:

A variable in Java has three components, which are listed below:

- **Data Type:** Defines the kind of data stored (e.g., int, String, float).
- **Variable Name:** A unique identifier following Java naming rules.
- **Value:** The actual data assigned to the variable.

Types of Java Variables

Now let us discuss different types of variables which are listed as follows:

- Local Variables
- Instance Variables
- Static Variables

1. Local Variables

A variable defined within a block or method or constructor is called a local variable.

- The Local variable is created at the time of declaration and destroyed when the function completed its execution.
- The scope of local variables exists only within the block in which they are declared.
- We first need to initialize a local variable before using it within its scope.

Example: This example shows how a **local variable** is declared and used inside the main method and it cannot be used outside of it.

```java
import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        // Declared a Local Variable
        int var = 10;

        // This variable is local to this main method only
        System.out.println("Local Variable: " + var);
    }
}
```

Output

Local Variable: 10

2. Instance Variables

Instance variables are known as non-static variables and are declared in a class outside of any method, constructor, or block.

- Instance variables are created when an object of the class is created and destroyed when the object is destroyed.
- Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used.
- Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable. For *String* it is *null*, for *float* it is *0.0f*, for *int* it is *0*, for Wrapper classes like *Integer* it is *null*, *etc.*
- Scope of instance variables are throughout the class except the static contexts.
- Instance variables can be accessed only by creating objects.
- We initialize instance variables using constructors while creating an object. We can also use instance blocks to initialize the instance variables.

Example: This example demonstrates the use of instance variables, which are declared within a class and initialized via a constructor, with default values for uninitialized primitive types.

```
class Geeks {

    // Declared Instance Variable
    public String geek;
    public int i;
    public Integer I;
    public Geeks()
    {
        // Default Constructor
        // initializing Instance Variable
        this.geek = "Sweta Dash";
    }

    // Main Method
    public static void main(String[] args)
    {
        // Object Creation
        Geeks name = new Geeks();

        // Displaying O/P
        System.out.println("Geek name is: " + name.geek);
        System.out.println("Default value for int is "+ name.i);

        // toString() called internally
        System.out.println("Default value for Integer is: "+ name.I);
    }
}
```

Output

Geek name is: Sweta Dash

Default value for int is 0

Default value for Integer is: null

3. Static Variables

Static variables are also known as class variables.

- These variables are declared similarly to instance variables. The difference is that static variables are declared using the static keyword within a class outside of any method, constructor, or block.

- Unlike instance variables, we can only have one copy of a static variable per class, irrespective of how many objects we create.

- Static variables are created at the start of program execution and destroyed automatically when execution ends.

- Initialization of a static variable is not mandatory. Its default value is dependent on the data type of variable. For *String* it is *null*, for *float* it is *0.0f*, for *int* it is *0*, for *Wrapper classes* like *Integer* it is *null*, etc.

- If we access a static variable like an instance variable (through an object), the compiler will show a warning message, which won't halt the program. The compiler will replace the object name with the class name automatically.

- If we access a static variable without the class name, the compiler will automatically append the class name. But for accessing the static variable of a different class, we must mention the class name as 2 different classes might have a static variable with the same name.

- Static variables cannot be declared locally inside an instance method.

- Static blocks can be used to initialize static variables.

Example: This example demonstrates the use of static variables, which belong to the class and can be accessed without creating an object of the class.

```java
import java.io.*;

class Geeks {

    // Declared static variable
    public static String geek = "Sweta Dash";

    public static void main(String[] args)
    {

        // geek variable can be accessed without object
        // creation Displaying O/P Geeks.geek --> using the
        // static variable
        System.out.println("Geek Name is: " + Geeks.geek);

        // static int c = 0;
        // above line, when uncommented,
        // will throw an error as static variables cannot be
        // declared locally.
    }
}
```

Output

Geek Name is: Sweta Dash

>> **Scope of Variables in Java**

The **scope of variables** is the part of the program where the variable is accessible. Like C/C++, in Java, all identifiers are lexically (or statically) scoped, i.e., scope of a variable can be determined at compile time and independent of the function call stack. In this article, we will learn about **Java Scope Variables**.

Java Scope of Variables

Java Scope Rules can be covered under the following categories.

- Instance Variables

- Static Variables
- Local Variables
- Parameter Scope
- Block Scope

1. Instance Variables – Class Level Scope

These variables must be declared inside a class (outside any function). They can be directly accessed anywhere in class. Let's take a look at an example:

```
public class Test {
// All variables defined directly inside a class
// are member variables
int a;
private String b;

void method1() {....}
int method2() {....}

char c;
}
```

- We can declare class variables anywhere in the class, but outside methods.
- Access specified of member variables doesn't affect scope of them within a class.
- Member variables can be accessed outside a class.

2. Static Variables – Class Level Scope

Static Variable is a type of class variable shared across instances. Static Variables are the variables which once declared can be used anywhere even outside the class without initializing the class. Unlike Local variables it scope is not limited to the class or the block.

```java
import java.io.*;

class Test{
    // static variable in Test class
    static int var = 10;
}

class Geeks
{
    public static void main (String[] args) {
        // accessing the static variable
        System.out.println("Static Variable : "+Test.var);
    }
}
```

Output

Static Variable : 10

3. Method Level Scope – Local Variable

Variables declared inside a method have method level scope and can't be accessed outside the method.

public class Test {
void method1()
{
// Local variable (Method level scope)
int x;
}
}

Note: Local variables don't exist after method's execution is over.

4. Parameter Scope – Local Variable

Here's another example of method scope, except this time the variable got passed in as a parameter to the method:

class Test {
private int x;

public void setX(int x) {
this.x = x;
}
}

5. Block Level Scope

A variable declared inside pair of brackets "{" and "}" in a method has scope within the brackets only.

```
public class Test
{
    public static void main(String args[])
    {
        // Block Level Scope
        {
            // The variable x has scope within
            // brackets
            int x = 10;
            System.out.println(x);
        }

        // Uncommenting below line would produce
        // error since variable x is out of scope.

        // System.out.println(x);
    }
}
```

Output

10

>> Type conversion in Assignments

Type conversion in Java refers to the process of changing a value from one data type to another. It's also known as type casting or type coercion. Type conversion in Java refers to the process of converting a variable from one data type to another. Java supports two main types of type conversion: implicit and explicit.

1. Widening Conversion (Implicit Type Casting)

In implicit type casting, the programming language automatically converts data from one type to another if needed. For example, if you have an integer variable and you try to assign it to a float variable, the programming language will automatically convert the integer to a float without you having to do anything.

Advantages:

- **Convenience:** Implicit casting saves time and effort because you don't have to manually convert data types.
- **Automatic:** It happens automatically, reducing the chance of errors due to forgetting to convert types.

Disadvantages:

1. **Loss of Precision:** Sometimes, when converting between data types, there can be a loss of precision. For example, converting from a float to an integer may result in losing the decimal part of the number.
2. **Unexpected Results:** Implicit casting can sometimes lead to unexpected results if the programmer is not aware of how the conversion rules work.

When to Use Implicit Type Casting

- Use implicit type casting when you are confident that the compiler will correctly determine the correct data type for a variable.
- Use implicit type casting when you are converting between compatible data types. For example, you can implicitly cast an integer to a double, but you cannot implicitly cast a double to an integer.

Integer to Floating Point:

```
int_num = 5
float_num = int_num  # Implicitly converts int to float
```

Smaller Data Type to Larger Data Type:

```
int num = 10;
double result = num;  // Implicitly converts int to double
```

Character to Integer:

```
char letter = 'A';
int ascii_value = letter;  // Implicitly converts char to int
```

Example

```
public class Main {
  public static void main(String[] args) {
    int myInt = 9;
    double myDouble = myInt; // Automatic casting: int to double

    System.out.println(myInt);      // Outputs 9
    System.out.println(myDouble);   // Outputs 9.0
  }
}
```

2. Narrowing Conversion (Explicit Type Casting)

Explicit type casting, also known as type conversion or type coercion, occurs when the programmer explicitly converts a value from one data type to another. Unlike implicit type casting, explicit type casting requires the programmer to specify the desired data type conversion.

Advantages:

- **Control:** Explicit type casting gives the programmer more control over the conversion process, allowing for precise manipulation of data types.
- **Clarity:** By explicitly indicating the type conversion, the code becomes more readable and understandable to other developers.
- **Avoidance of Loss of Precision:** In cases where precision is crucial, explicit type casting allows the programmer to handle the conversion carefully to avoid loss of precision.

Disadvantages:

- **Complexity:** Explicit type casting can introduce complexity to the code, especially when dealing with multiple data type conversions.
- **Potential Errors:** If the programmer incorrectly performs explicit type casting, it may result in runtime errors or unexpected behaviour.
- **Additional Syntax:** Explicit type casting requires additional syntax or function calls, which may increase code verbosity.

When to Use Explicit Type Casting

- Use explicit type casting when you want to ensure that a variable is converted to a specific data type.
- Use explicit type casting when you are converting between incompatible data types. For example, you can explicitly cast a double to an integer, but you cannot implicitly cast an integer to a double.

Example: write a simple java program to demonstrate explicit type conversion

```java
public class ExplicitTypeConversion {
    public static void main(String[] args) {
        double doubleVal = 123.456;    // Double value
        float floatVal = (float) doubleVal; // double to float (explicit)
        long longVal = (long) floatVal;     // float to long (explicit)
        int intVal = (int) longVal;         // long to int (explicit)
        short shortVal = (short) intVal;    // int to short (explicit)
        byte byteVal = (byte) shortVal;     // short to byte (explicit)

        System.out.println("Double value: " + doubleVal);
        System.out.println("Float value (double to float): " + floatVal);
        System.out.println("Long value (float to long): " + longVal);
        System.out.println("Int value (long to int): " + intVal);
        System.out.println("Short value (int to short): " + shortVal);
        System.out.println("Byte value (short to byte): " + byteVal);
    }
}
```

Output:

```
Double value: 123.456
Float value (double to float): 123.456
Long value (float to long): 123
Int value (long to int): 123
Short value (int to short): 123
Byte value (short to byte): 123
```

>> Precedence and Associativity of Java Operators

Precedence and associative rules are used when dealing with hybrid equations involving more than one type of operator. In such cases, these rules determine which part of the equation to consider first, as there can be many different valuations for the same equation. The below table

depicts the precedence of operators in decreasing order as magnitude, with the top representing the highest precedence and the bottom showing the lowest precedence.

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ~ ! (type)	Right to left	Unary prefix
* / %	Left to right	Multiplicative
+ -	Left to right	Additive
<< >> >>>	Left to right	Shift
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
|	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
||	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment

Example: write a simple java program to demonstrate precedency and associativity of arithmetic operators in java

```
public class OperatorPrecedence {

    public static void main(String[] args) {

        // Precedence example

        int result1 = 10 + 5 * 2;

        System.out.println("Result of 10 + 5 * 2 = " + result1); // Expected: 20

        // Using parentheses to change precedence
```

```
        int result2 = (10 + 5) * 2;
        System.out.println("Result of (10 + 5) * 2 = " + result2); // Expected: 30
        // Associativity example (left-to-right for + and -)
        int result3 = 20 - 5 - 2;
        System.out.println("Result of 20 - 5 - 2 = " + result3); // Expected: 13
        // Associativity example (left-to-right for /)
        int result4 = 100 / 10 / 2;
        System.out.println("Result of 100 / 10 / 2 = " + result4); // Expected: 5
        // Mixed example
        int result5 = 4 + 3 * 2 - 1;
        System.out.println("Result of 4 + 3 * 2 - 1 = " + result5); // Expected: 9
    }
}
```

Output:

```
Result of 10 + 5 * 2 = 20
Result of (10 + 5) * 2 = 30
Result of 20 - 5 - 2 = 13
Result of 100 / 10 / 2 = 5
Result of 4 + 3 * 2 - 1 = 9
```